

Clasificador de Señales en Tiempo Real sobre STM32 utilizando FreeRTOS y NanoEdge AI

Matías Oliva

30 de junio de 2026

Índice

1. Introducción	2
2. Objetivos	2
3. Hardware Utilizado	2
4. Arquitectura General	3
5. Interface de usuario (tarea StartUartTask)	3
6. Generación de Señales (tarea StartDacTask)	3
6.1. Control del DAC	4
7. Adquisición de Datos (tarea StartAdcTask)	4
7.1. Half Transfer y Transfer Complete	4
8. Dataset para Entrenamiento	5
9. Entrenamiento con NanoEdge AI Studio	5
10. Implementación del Modelo en STM32	6
11. Migración a FreeRTOS	7
12. Aplicación de Monitoreo en Python	7
13. Resultados	8
14. Conclusiones	9
15. Trabajo Futuro	9

1. Introducción

Este informe describe el desarrollo de un sistema embebido capaz de generar, adquirir y clasificar señales en tiempo real utilizando una placa STM32 NUCLEO-U575ZI-Q. El proyecto combina técnicas de procesamiento digital de señales, sistemas operativos de tiempo real y aprendizaje automático embebido mediante NanoEdge AI Studio. El objetivo principal es identificar automáticamente diferentes formas de onda generadas por el propio sistema utilizando un modelo de clasificación ejecutado localmente en el microcontrolador.

El código es abierto y está disponible en:

- https://github.com/ushikawa93/signal_classifier_with_rtos_stm32.

2. Objetivos

- Generar señales sinusoidales, cuadradas y triangulares utilizando el DAC.
- Adquirir las señales mediante el ADC.
- Construir un dataset para entrenamiento.
- Entrenar un clasificador utilizando NanoEdge AI Studio.
- Implementar el modelo en un STM32.
- Utilizar FreeRTOS para coordinar las distintas tareas del sistema.
- Desarrollar una interfaz gráfica para monitoreo y captura de datos.

3. Hardware Utilizado

- STM32 NUCLEO-U575ZI-Q
- DAC interno
- ADC interno
- Comunicación UART
- Cable de conexión DAC-ADC

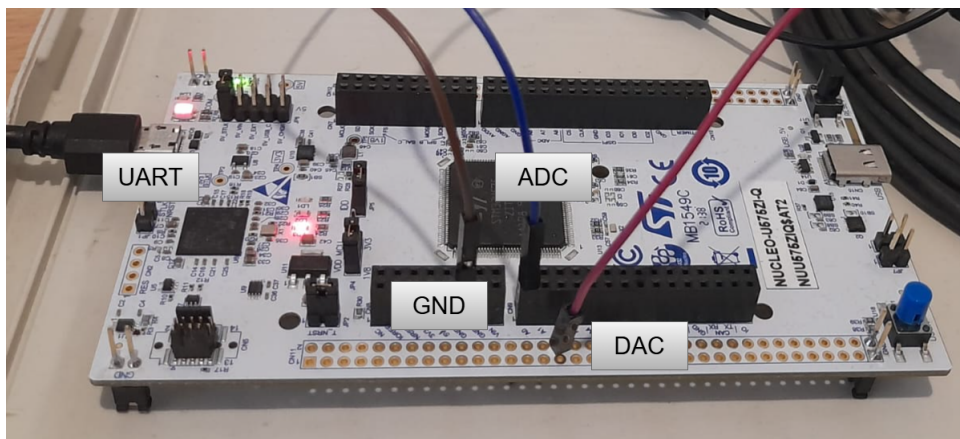


Figura 1: Montaje experimental

4. Arquitectura General

La arquitectura implementada se basa en la generación de señales mediante DAC, adquisición mediante ADC y posterior clasificación utilizando un modelo de Machine Learning.

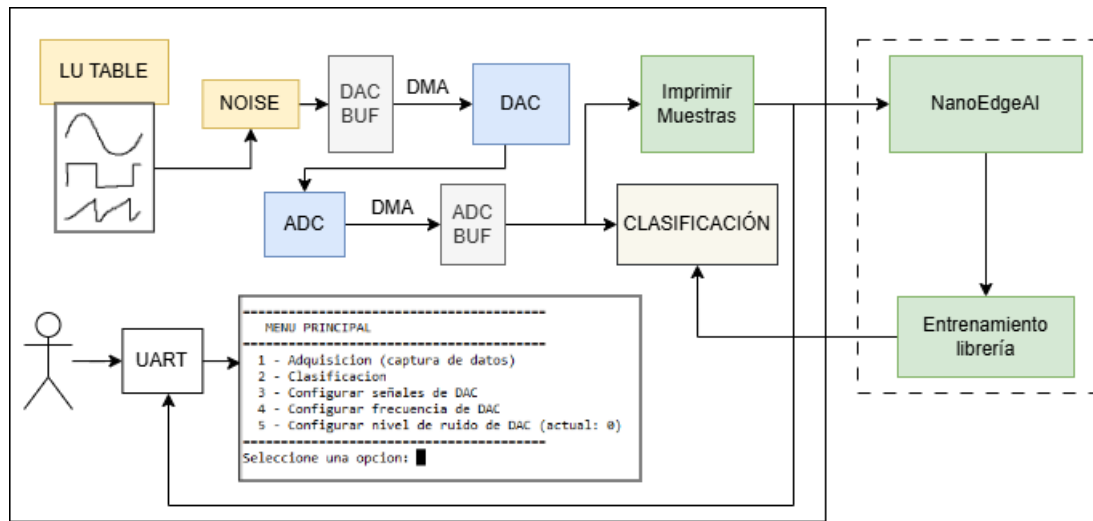


Figura 2: Arquitectura general

5. Interface de usuario (tarea StartUartTask)

A través del puerto UART el sistema presenta al usuario un menú definido en el header `menú.h`. En este se le presentan las opciones:

- 1 - Adquisición (captura de datos)
- 2 - Clasificación
- 3 - Configurar señales de DAC
- 4 - Configurar frecuencia de DAC
- 5 - Configurar nivel de ruido de DAC

La operación del menú y su derivación a las tareas correspondientes están controladas por la tarea `StartUartTask`.

6. Generación de Señales (tarea StartDacTask)

La generación de señales se implementó utilizando:

- DAC interno (12 bits).
- DMA en modo circular.
- Temporizador como trigger.
- Tablas precalculadas de muestras.
- Generador de ruido interno del STM32.

Las formas de onda implementadas son:

- Sinusoidal.
- Cuadrada.
- Triangular.

6.1. Control del DAC

La tarea **StartDacTask** (definido en **app_freertos.h**) junto con el header **signals.h** controlan la operación del DAC.

Esta tarea comienza agregándole ruido a las LU tables (**generateSignalsWithNoise(int)**) y luego lanza el DAC por DMA. Este esquema tiene la limitación de que el ruido se calcula solo en este punto, por lo que todos los ciclos de la señal tienen los mismos esquemas de ruido (es decir, el ruido es periódico). En un futuro podría mejorarse este punto recalculando las tablas en alguna interrupción que se de al finalizar el buffer del DMA.

La transferencia memoria- DAC es disparada por el temporizador TIM2. Por defecto este timer esta configurado para generar una señal de 1 segundo de periodo, a partir de una LU table de 128 puntos (triggers cada 1/128 s).

Esta tarea esta atenta a tres eventos que pueden dispararse desde la UART que permiten modificar la forma de onda:

- **DAC_WAVE**: Cambio de la forma de onda.
- **DAC_NOISE**: Cambia en nivel de ruido en la señal. Para ello detiene el DAC y vuelve a llamar a **generateSignalsWithNoise(int)**. El nivel de ruido se puede configurar de 0 a 4095 (cuentas del DAC).
- **DAC_FREQ**: Cambia la frecuencia de actualización del DAC, llamando a la función **getTIMER_DAC(int)** para calcular el periodo del timer deseado a partir de la frecuencia de señal objetivo.

7. Adquisición de Datos (tarea **StartAdcTask**)

La adquisición se realiza mediante el ADC utilizando DMA circular. Su ejecución esta definida en la tarea **StartAdcTask**. Esta configura la adquisición por DMA al principio de la operación y luego queda esperando eventos que señalizan bloques de datos listos para procesar.

Para ello se implementó un buffer doble de 256 muestras para permitir el procesamiento simultáneo de los datos mientras continúa la adquisición. La transacción del DMA es disparada por el TIM15, configurado para funcionar a 100 Hz (frecuencia de muestreo).

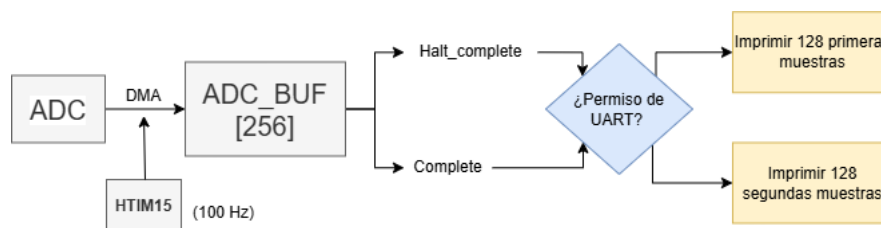


Figura 3: Configuración ADC-DMA

7.1. Half Transfer y Transfer Complete

A medida que se llena el buffer del ADC se disparan dos interrupciones que le avisan a la tarea **StartAdcTask** que hay datos disponibles para procesar

- **Half Complete.** Las primeras 128 muestras del buffer están disponibles
- **Complete.** Las segundas 128 muestras del buffer están disponibles.

En esta etapa del desarrollo del proyecto el procesamiento de los datos consiste simplemente en imprimir cierta cantidad de bloques a través del puerto UART cuando el usuario lo solicita.

8. Dataset para Entrenamiento

Para entrenar el clasificador se implementó un modo de captura que transmite muestras por UART en formato JSON.

Ejemplo:

```
{
  "ts": "00:00:00",
  "Muestras": [...]
}
```

Cuando el usuario lo pide se imprimen por puerto UART muestras en este formato. Estos se pueden almacenar y utilizar para entrenar algoritmos de clasificación utilizando NanoEdge AI Studio.

```
=====
MENU PRINCIPAL
=====
1 - Adquisicion (captura de datos)
2 - Clasificacion
3 - Configurar señales de DAC
4 - Configurar frecuencia de DAC
5 - Configurar nivel de ruido de DAC (actual: 0)
=====
Seleccione una opcion: 1
Ingrese numero de datos a adquirir ('c' para adquisicion continua)...
1
{ "ts":00:00:00 , "Muestras": 6483, 6854, 7289, 7621, 8034, 8414, 8864, 9218, 962
5, 10008, 10404, 10728, 11159, 11498, 11917, 12241, 12532, 12889, 13150, 13500, 13
783, 14047, 14398, 14564, 14806, 15014, 15221, 15432, 15584, 15679, 15840, 15970,
16020, 16116, 16142, 16175, 16217, 16211, 16169, 16113, 16021, 15960, 15835, 15750
, 15609, 15429, 15237, 14923, 14837, 14716, 14378, 14109, 13830, 13520, 13092, 129
38, 12609, 12281, 11936, 11564, 11186, 10840, 10462, 10075, 9695, 9282, 8894, 8509
, 8104, 7733, 7299, 6921, 6524, 6148, 5784, 5379, 5023, 4654, 4335, 3964, 3646, 32
54, 2985, 2702, 2401, 2145, 1882, 1644, 1406, 1223, 977, 775, 645, 491, 380, 248,
215, 168, 40, 42, 131, 44, 76, 137, 168, 277, 344, 490, 674, 782, 977, 1110, 1298,
1684, 1839, 2126, 2355, 2555, 2861, 3270, 3646, 3888, 4152, 4590, 4954, 5277, 579
0, 6063, }
```

Figura 4: Captura de datos para entrenamiento

9. Entrenamiento con NanoEdge AI Studio

Utilizando la plataforma se adquirieron señales de distintas frecuencias, sinusoidales, ondas cuadradas y triangulares, que fueron utilizadas para entrenar un modelo de clasificación en NanoEdge AI Studio. Para adaptar las señales al formato csv requerido por el software se utilizaron scripts disponibles en la carpeta /utils del proyecto. En esta carpeta también hay disponible un script de matlab para graficar segmentos de las señales obtenidas y verificar su integridad.

- **Dataset utilizado:** Los datos específicos que se utilizaron para entrenar el modelo se encuentran en la carpeta nanoEdgeAI/dat.

- **Número de clases:** El modelo se configuró con tres clases: *OndaSinusoidal*, *OndaCuadrada* y *OndaTriangular*.
- **Configuración del entrenamiento:** Se empleó la versión 5.1.1 de NanoEdge AI Studio sobre la plataforma NUCLEO-U575ZI-Q, con señales de 128 columnas por muestra. El dataset fue balanceado con 3 archivos de 60 muestras por archivo en cada clase (180 líneas por clase). La búsqueda de la librería óptima se realizó mediante el benchmark automático (*autoML*) de la herramienta, evaluando 18199 librerías candidatas durante 43 minutos con la configuración por defecto.
- **Resultados obtenidos:** El benchmark finalizó seleccionando una librería con un *Quality Index* de 99/100 puntos y una *Balanced Accuracy* del 100 %, con un consumo de memoria de 1.7 KB de RAM (incluyendo buffer de 0.6 KB) y 7.8 KB de Flash. La matriz de confusión sobre el conjunto de validación no presentó ningún error de clasificación entre las tres clases (36 muestras correctamente clasificadas por clase), tal como se muestra en la Figura 5.

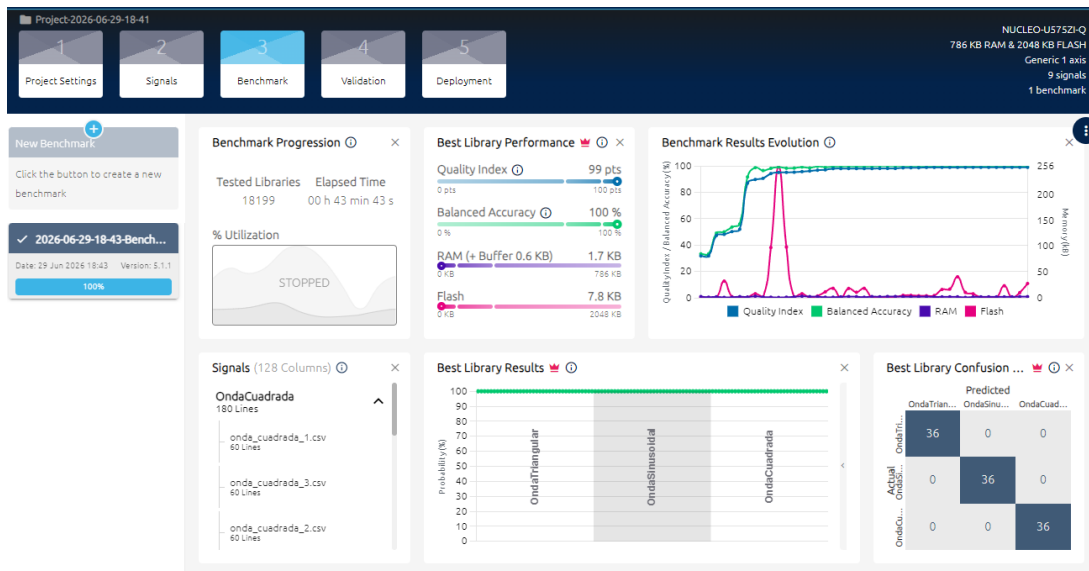


Figura 5: Resultados finales del benchmark en NanoEdge AI Studio.

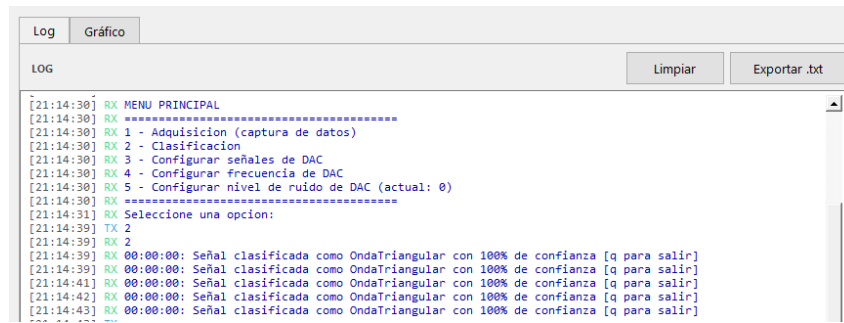
10. Implementación del Modelo en STM32

El modelo generado fue integrado al firmware mediante las bibliotecas proporcionadas por NanoEdge AI Studio.

El modelo se inicializa antes de lanzar las tareas, mediante la función `neai_classification_init()` provista en la librería.

Luego, dentro de la tarea que controla el UART en la librería, se agregó la clasificación de señales. En este punto, cuando es requerido por el usuario se utiliza la función `neai_classification()`, pasándole el contenido del buffer proveniente del ADC que este completo en ese momento (puede ser la primera o la segunda mitad del ADC_BUF). Antes de utilizarse directamente las muestras se convierten de entero a punto flotante, por requerimientos de la librería.

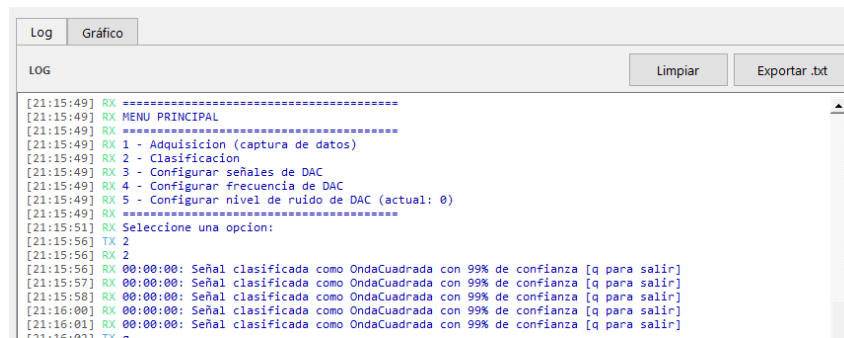
Usando este esquema el sistema decide entre alguna de las tres ondas lo que está muestreando en el ADC, y expone los resultados junto con el nivel de similaridad obtenido.



```
Log Gráfico
LOG
Limpiar Exportar .txt

[21:14:30] RX MENU PRINCIPAL
[21:14:30] RX =====
[21:14:30] RX 1 - Adquisicion (captura de datos)
[21:14:30] RX 2 - Clasificacion
[21:14:30] RX 3 - Configurar señales de DAC
[21:14:30] RX 4 - Configurar frecuencia de DAC
[21:14:30] RX 5 - Configurar nivel de ruido de DAC (actual: 0)
[21:14:30] RX =====
[21:14:31] RX Seleccione una opcion:
[21:14:39] TX 2
[21:14:39] RX 2
[21:14:39] RX 00:00:00: Señal clasificada como OndaTriangular con 100% de confianza [q para salir]
[21:14:39] RX 00:00:00: Señal clasificada como OndaTriangular con 100% de confianza [q para salir]
[21:14:41] RX 00:00:00: Señal clasificada como OndaTriangular con 100% de confianza [q para salir]
[21:14:42] RX 00:00:00: Señal clasificada como OndaTriangular con 100% de confianza [q para salir]
[21:14:43] RX 00:00:00: Señal clasificada como OndaTriangular con 100% de confianza [q para salir]
```

(a) Clasificador ante una onda triangular



```
Log Gráfico
LOG
Limpiar Exportar .txt

[21:15:49] RX =====
[21:15:49] RX MENU PRINCIPAL
[21:15:49] RX =====
[21:15:49] RX 1 - Adquisicion (captura de datos)
[21:15:49] RX 2 - Clasificacion
[21:15:49] RX 3 - Configurar señales de DAC
[21:15:49] RX 4 - Configurar frecuencia de DAC
[21:15:49] RX 5 - Configurar nivel de ruido de DAC (actual: 0)
[21:15:49] RX =====
[21:15:51] RX Seleccione una opcion:
[21:15:56] TX 2
[21:15:56] RX 2
[21:15:56] RX 00:00:00: Señal clasificada como OndaCuadrada con 99% de confianza [q para salir]
[21:15:57] RX 00:00:00: Señal clasificada como OndaCuadrada con 99% de confianza [q para salir]
[21:15:58] RX 00:00:00: Señal clasificada como OndaCuadrada con 99% de confianza [q para salir]
[21:16:00] RX 00:00:00: Señal clasificada como OndaCuadrada con 99% de confianza [q para salir]
[21:16:01] RX 00:00:00: Señal clasificada como OndaCuadrada con 99% de confianza [q para salir]
```

(b) Clasificador ante una onda cuadrada

Figura 6: Capturas de pantalla del proceso de clasificación

11. Migración a FreeRTOS

Inicialmente el sistema fue desarrollado utilizando una arquitectura secuencial, con modos configurados a través de `#defines`. Esta implementación preliminar está disponible en:

- https://github.com/ushikawa93/signal_classifier_stm32

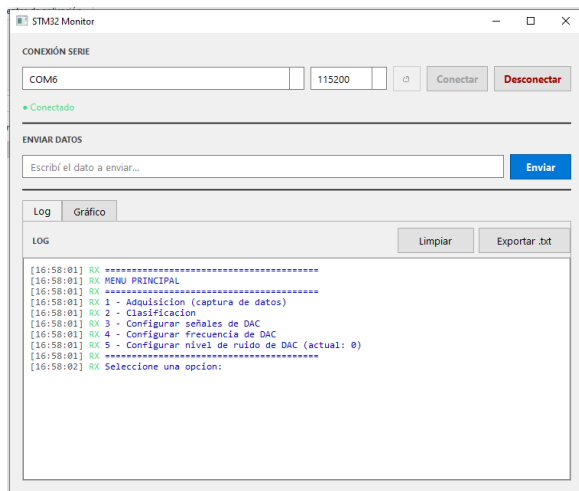
Posteriormente se migró a FreeRTOS para mejorar la organización del software y desacoplar las distintas funcionalidades.

12. Aplicación de Monitoreo en Python

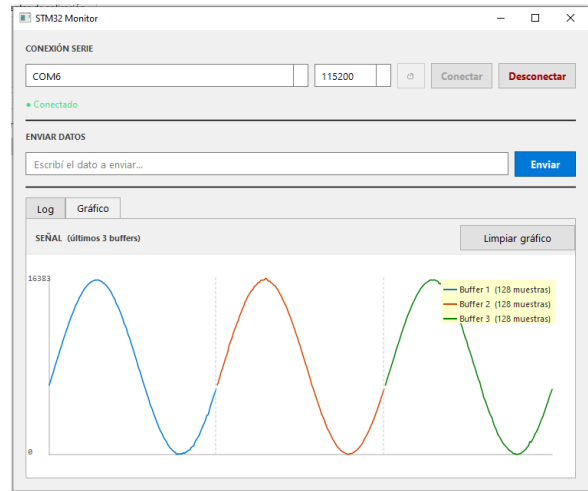
Se desarrolló una aplicación de escritorio utilizando PySide6 para interactuar con el sistema. Las funcionalidades implementadas incluyen:

- Conexión serie.
- Visualización de mensajes UART.
- Captura de buffers.
- Graficación en tiempo real.
- Exportación de logs.

Esta implementación está disponible en `/python_gui`, con un ejecutable ubicado en `/python_gui/dist/main.exe`



(a) Vista de consola



(b) Vista de señales

Figura 7: Aplicación de Monitoreo en Python

13. Resultados

Para evaluar la robustez del sistema de clasificación ante condiciones de ruido, se realizaron 5 repeticiones de un test sistemático que consistió en clasificar cada tipo de señal (sinusoidal, cuadrada y triangular) bajo niveles crecientes de ruido: 0, 100, 200, 500, 1000, 2000 y 3000 (en unidades DAC). Los resultados obtenidos se resumen en la Tabla 1.

Cuadro 1: Resultados promedio de clasificación por señal y nivel de ruido (5 repeticiones)

Señal	Ruido	Aciertos/5	Confianza prom.	Resultado
Sinusoidal	0	5/5	100 %	✓
Sinusoidal	100	5/5	99 %	✓
Sinusoidal	200	5/5	100 %	✓
Sinusoidal	500	5/5	100 %	✓
Sinusoidal	1000	5/5	100 %	✓
Sinusoidal	2000	5/5	98 %	✓
Sinusoidal	3000	5/5	98 %	✓
Cuadrada	0	4/5	81 %	✓
Cuadrada	100	5/5	93 %	✓
Cuadrada	200	4/5	85 %	✓
Cuadrada	500	5/5	100 %	✓
Cuadrada	1000	4/5	99 %	✓
Cuadrada	2000	2/5	98 %	×
Cuadrada	3000	0/5	99 %	×
Triangular	0	4/5	99 %	✓
Triangular	100	5/5	99 %	✓
Triangular	200	5/5	98 %	✓
Triangular	500	5/5	99 %	✓
Triangular	1000	5/5	100 %	✓
Triangular	2000	5/5	96 %	✓
Triangular	3000	5/5	84 %	✓

La señal sinusoidal fue clasificada correctamente en el 100 % de los casos independientemente del nivel de ruido, manteniendo una confianza promedio superior al 98 % en todos los niveles

evaluados. La señal triangular presentó un comportamiento igualmente robusto, con un único error de clasificación observado en el test 1 con ruido nulo, posiblemente debido a condiciones transitorias al inicio del ensayo.

La señal cuadrada resultó ser la más sensible al ruido. Para niveles de hasta 1000 se obtuvo una tasa de aciertos aceptable, mientras que a partir de ruido 2000 el modelo comenzó a confundirla con la onda triangular, y con ruido 3000 la clasificación fue sistemáticamente incorrecta en los 5 tests, siendo identificada como onda sinusoidal en 4 de ellos. Este comportamiento es esperable dado que con ruido suficientemente elevado los flancos abruptos característicos de la onda cuadrada se suavizan, asemejando el espectro de otras formas de onda.

En términos generales, el modelo demostró una alta capacidad de clasificación para niveles de ruido de hasta 1000, con una tasa de aciertos global del 93 % en ese rango. Por encima de ese umbral, el desempeño se degrada principalmente en la clase cuadrada.

14. Conclusiones

El proyecto permitió ganar experiencia con la herramienta NanoEdgeAI y el entorno de desarrollo de STM32. El resultado final integra distintas tecnologías dentro de una única plataforma embebida:

- STM32.
- DMA.
- FreeRTOS.
- NanoEdge AI.
- Comunicación UART.
- Aplicaciones de escritorio en Python.

En la versión actual el sistema permite generar adquirir y clasificar señales utilizando recursos limitados de procesamiento y constituye un punto de partida sólido para el trabajo futuro en estas plataformas. El sistema todavía tiene muchos puntos a mejorar pensando en su aplicación futura a temas de monitoreo atmosférico, que se detallan en la siguiente sección.

15. Trabajo Futuro

- Mejorar el método de agregado de ruido del sistema.
- Incorporar señales externas.
- Agregar nuevas clases a la clasificación.
- Investigar la causa de la baja robustez de la clase *OndaCuadrada* ante niveles de ruido elevados. En este sentido puede considerarse reentrenar el sistema con datos ruidosos.
- Entrenar un algoritmo de detección de anomalías en las señales.
- Probar el sistema con datos reales de sensores.
- Optimizar el consumo de RAM y Flash de la librería generada, evaluando el trade-off entre tamaño del modelo y exactitud.
- Validar el sistema en un rango más amplio de frecuencias de muestreo y adquisición.